UNIVERSITY OF BUCHAREST

FACULTY OF
MATHEMATICS AND
COMPUTER SCIENCE

SPECIALIZATION ARTIFICIAL INTELLIGENCE

Master Thesis

A POST-HOC ANALYSIS OF XLSTM ARCHITECTURES

GRADUATE

Tănase Victor-Flavian

SCIENTFIC COORDINATOR

Cristian Rusu PhD

Bucharest, July 2026

Abstract

Recent progress in natural language processing has been largely driven by large language models(LLMs). These architectures aim to learn representations of text that capture syntax, semantics, and long-range dependencies in an efficient and scalable way. Despite significant advances in performance, the internal mechanisms by which these models store and manipulate information remain only partially understood.

This limitation becomes especially relevant in deep models that rely on fundamentally different approaches to information storage. Transformer-based models process sequences using self-attention, where information is mixed across tokens at each layer without an explicit notion of persistent memory. In contrast, the xLSTM, a recently developed architecture was proposed by Beck et al. [4] as an alternative to the transformer architectures by extending the classic LSTM cell and reviving the interest in Recurrent Neural Networks

This work performs an analysis of the xLSTM architecture through a set of interpretability experiments aimed at understanding how information is represented, stored, and transformed inside the model. Moreover, the practicality of this architecture is assessed by evaluating it in a question answering setup, a very common application of LLMs.

We track how the model's intermediate hidden states evolve and compare them to those produced by a Transformer model to assess the similarity of their internal representations. Furthermore, we inspect memory behavior during the processing of sequences to understand how individual tokens influence the retention and discarding information.

Finally, we evaluate memory caching as a practical use of xLSTM's fixed-size memory state. By reusing the memory state after processing the context, the model avoids recomputing the input sequence and achieves inference speedups that increase with context length. We compare this approach against Mistral-7B and examine the trade-off between inference efficiency and task performance.

Sinopsis

Progresul recent in prelucrarea limbajului natural a fost accelerată de dezvoltarea modelelor lingvistice de mari dimensiuni(LLM). Acest tip de arhitectura are ca scop învățarea unor reprezentări ale limbajului care pot captura proprietăți sintactice, semantice și să modele dependențe între cuvinte îndepărtate într-un mod efficient și scalabil. În ciuda îmbunătățirii performanței acestor modele, modul in care ele învață și manipulează informația este doar parțial înțeles.

Această limitare este cu atât mai relevantă în arhitecturile moderne care se bazează pe abordări fundamental diferite de a reprezenta informația. Modelele de tip transformer procesează secvențe de text cu ajutorul mecanismului de atenție, care folosește informația din tokeni în fiecare strat intern, fără o structură definită pentru memorie. Pe de altă parte, xLSTM o arhitectură de tip recurentă[4] reprezintă o alternativă la modelele de tip transformer, folosind o structură pentru memorie explicită, care este actualizată incremental pe parcursul procesării textului. Acest mecanism are ca avantaj înmagazinarea si rafinarea informației de fiecare dată când un nou token este procesat.

Această lucrare investighează comportamentul arhitecturilor xLSTM printr-o serie de experimente care au ca scop să îmbunătățească modul în care sunt interpretate reprezentarea, înmagazinarea și transformările datelor în interiorul modelului. Analiza se desfășoară pe mai multe nivele de abstractizare, începând de la analiza la nivel de token, ca mai apoi să fie observate reprezentările la nivel de strat. Mai specific, sunt utilizate tehnici de tip logit lens[19] [2] pentru a remarca evoluția relației dintre reprezentările intermediare în spațiul de embeddings și spațiul vocabularului. De asemenea, comparăm reprezentările interne ale modelului xLSTM cu cele ale unei arhitecturi comparabile de tip transformer, anume Mistral-7B[16]. Scopul comparației este de a observa similaritatea evoluției reprezentărilor semantice dintre două arhitecturi diferite. Totodată, analizăm modificarea memoriei pe parcursul procesării tokenilor, urmărind evoluția memoriei pe parcursul parcurgerii unui text. și urmărim activările porților input și forget la nivel de token pentru a înțelege cum modelul alege sa rețină și să elimine selectiv conext. Împreună aceste experimente oferă o perspectivă nuanțată a interpretării mecanismelor din interiorul arhitecturii xLSTM.

În final, explorăm o utilizare practică a stării de memorie de dimensiune fixă din xLSTM prin memorarea informației contextuale în scenarii care testează abilitatea modelului de a răspunde la întrebări. Această abordare permite îmbunătățirea vitezei de răspuns pe măsură ce dimensiunea contextului crește, deoarece starea de memorie stocată poate fi reutilizată fără a procesa întreaga secvență de intrarară această metodă cu modelul Mistral 7B și analizăm avantajele și dezavantajele arhitecturilor recurente și celor bazate pe Transformere din perspectiva vitezei și a performanței de fiecare dată.

Contents

1 Introduction

Since the inception of the Transformer [33], large architectures utilizing stacked attention mechanisms have dominated the research landscape, exhibiting impressive performance in natural language processing tasks. This architectural paradigm scaled rapidly, progressing from hundreds of millions of parameters to billions, and ultimately to trillions for enterprise-scale models. In parallel, with this scaling trend, a significant part of the scientific community focused their efforts on understanding the internal mechanisms of these models. However, this intense scientific work has focused almost exclusively towards attention-based architectures, leaving non-transformer based models comparatively less explored.

Despite their success, Transformers suffer from a fundamental computational bottleneck: the self-attention mechanism scales quadratically ($\mathcal{O}(N^2)$) with respect to sequence length, making long-context processing computationally expensive. This limitation has led to a recent resurgence of interest in recurrent and state-based architectures that aim to achieve linear-time efficiency ($\mathcal{O}(N)$) without sacrificing performance. Key contributions in this direction include: Structured State Space Models (SSMs) like Mamba [13], linear transformers such as Receptance Weighted Key Value (RWKV)[23] and Extended Long-Short Term Memory(xLSTM) [4].

The xLSTM architecture extends the classic LSTM cell [15] by introducing exponential gating and matrix-valued memory states with gated updates. These changes look to address the scaling problem of the Transformers and the storage bottlenecks of traditional LSTMs, and try to achieve comparable perfromance while maintaining linear-time inference.

1.1 Problem

Deep architectures have produced tremendous advancements in the field of machine learning over the past decade. Their pattern recognition and information representation capabilities are yet to be completely understood even to this day. Post-hoc interpretability in deep neural networks is a challenging problem due to several intrinsic mathematical and structural properties of these models.

First, modern architectures operate in high-dimensional representation spaces, where features are distributed across multiple dimensions. Moreover, multiple concepts may be encoded within overlapping directions of the representation space.

Second, neural networks apply a sequence of non-linear transformations, making the relationship between input tokens and internal representations difficult to describe. As network depth increases, interactions between representations become progressively more complex.

Finally, although the computation of hidden states is fully specified by the model parameters and architecture, the resulting computation graph is generally too complex to provide specific human-interpretable explanations of how particular representations or predictions are produced.

Beyond interpretability challenges, recurrent-based architectures such as xLSTM also introduce important systems-level considerations. In contrast to purely attention-based models, recurrent models maintain an evolving internal state that can, in principle, reduce the computational overhead associated with processing long contexts. However, in practice, the models suffer from significant generation latency due to the sequential nature of the RNNs. This makes efficient inference an important bottleneck to be addressed, particularly when dealing with long sequences or real-time constraints.

1.2 Motivation

The motivation for this work originates from the recent resurgece of recurrent architectures in large-scale sequence modelling. While analyizing the current state of research on recurrent neural networks, the xLSTM architecture, which Beck et al.[4] proposed in 2024, became a personal point of interest. As one of the first recurrent architectures in recent years to demonstrate competitive performance at scales traditionally dominated by Transformer-based models, xLSTM represents an important development in the ongoing search for efficient alternatives to self-attention.

While the performance of Transformer-based language models has been extensively studied, a substantial body of interpretability research has also emerged to investigate their internal mechanisms. In contrast, the behavior of modern recurrent architectures such as xLSTM remains comparatively underexplored. This observation motivated the present work, which applies and adapt interpretability techniques inspired by previous studies of neural language models.

By investigating how information is represented, stored, and updated within xLSTM, this thesis aims to contribute to a better understanding of the model's memory mechanisms and internal representations. Such insights may help validate architectural design choices, identify limitations, and inform future developments of memory-based language models.

At the same time, it is important to evaluate how alternative architectures perform in practice compared to Transformers, which are widely used and benefit from highly optimized inference systems. This motivates the study of xLSTM under realistic inference conditions.

1.3 Contribution

To address the lack of interpretability analyses for scalable recurrent architectures, the main objective of this thesis is to perform a post-hoc evaluation of the internal representations and state dynamics of the xLSTM-7B architecture. This work analyzes the internal mechanisms of the model by first studying the hidden states and then examining the individual components of the xLSTM cells that contribute to the hiden state values.

The first phase of this work establishes an empirical baseline by evaluating the pretrained xLSTM-7B model across standard datasets, specifically LAMBADA (OpenAI)[21], PIQA[6], Winogrande[28], ARC-Challenge[9], ARC-Easy[9], and Hellaswag[36]. This evaluation represents an assessment of the model's capablities before analyzing its internal structure. Following this initial evaluation, the second phase adapts the logit lens[19] [2] to project intermediate hidden states from the recurrent layers directly onto the vocabulary space.

Next, we compare how hidden representations evolve across the layers of a recurrent network compared to a Transformer, using xLSTM-7B and Mistral-7B. The final two phases of the interpretability study focus on the sequential processing of the architecture. Specifically, we examine the evolution of the memory state over time and the contribution of individual tokens to long-term memory through the input and forget gates. To analyze these mechanisms, we process sequences and track the norm of the cell state, together with the activations of the input and forget gates. This allows us to observe how information is stored, retained, and updated throughout the sequence.

On top of the interpretability experiments, we evaluate the practical utility of the xLSTM model in a question answering setting. We leverage the constant-size memory state of the model and explore caching it when processing context that is later reused for answering questions. This approach enables an inference speedup that increases with the size of the context, as the stored memory can be reused without recomputing the full sequence.

1.4 Outline

This thesis is structured as follows. Section 2 presents the theoretical background underlying the work. It introduces Recurrent Neural Networks, Long Short-Term Memory cells, and the Extended Long Short-Term Memory architecture.

Section 3 describes the methodology used in the experiments. More specifically, it outlines the research objectives, the models considered, and the experimental setup, including the theoretical formulation of the conducted experiments. The technologies and implementation details are presented in Section 4.

Section 5 presents and discusses the experimental results. Finally, conclusions, limitations, and directions for future work are provided in Section **??**.

2 Preliminaries and Related Work

To provide the necessary background for the evaluation of the xLSTM-7B architecture, this section covers the theoretical concepts and related work relevant to this study.

2.1 Recurrent Neural Networks

Recurrent Neural Networks (RNNs) represent one of the earliest neural architectures specifically designed for sequence modeling tasks. Their history can be traced to the work of Rumelhart et al. [27], who introduced the backpropagation algorithm and discussed neural networks with recurrent connections. Unlike feedforward neural networks, which process each input independently, RNNs maintain a hidden state that evolves over time, allowing information from previous inputs to influence future predictions.

The key idea behind recurrent architectures is the reuse of the same set of parameters at every time step. Given an input sequence, the hidden state is updated recursively based on both the current input and the previous hidden state, enabling the network to capture temporal dependencies within the data. This property made RNNs a natural choice for tasks involving sequential information, such as language modeling, speech recognition, machine translation, and time-series forecasting.

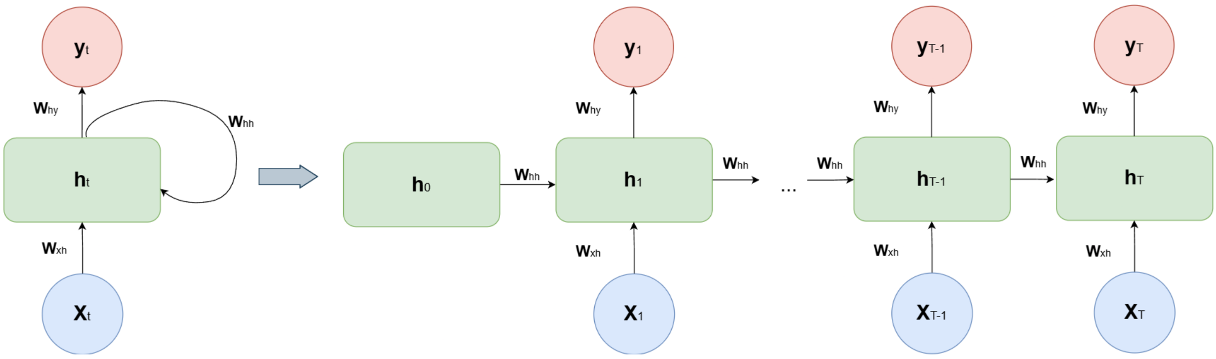


Figure 1: RNN: compressed (left) and unfolded(right), diagram inspired by [20]

The operation of a standard RNN can be formally described through its hidden state updat in equation (1). At a given timestamp $t = \overline{1,T}$, for an input x_t, the hidden state h_t is computed as:

$$h_t = \phi(W_{xh}x_t + W_{hh}h_{t-1} + b_h), \tag{1}$$

where W_{xh} and W_{hh} are learnable weight matrices, b_h is a bias term, and $\phi(\cdot)$ denotes a non-linear activation function such as tanh or sigmoid. The hidden state is then used to produce the output y_t:

$$y_t = W_{hy}h_t + b_y, \tag{2}$$

where W_{hy} and b_y are the output weight matrix and bias, respectively.

Through the recurrent connection involving h_{t-1}, the network is able to propagate information across time steps, allowing previous inputs to influence future predictions.

Training recurrent neural networks requires propagating gradients not only through the layers of the network but also through time. This is typically achieved using *Backpropagation Through Time* (BPTT), introduced by Werbos et al. [34]. However, this training procedure suffers from instability issues, namely *vanishing* and *exploding* gradients [5].

To mitigate the exploding gradient problem, Pascanu et al. [22] introduced *gradient clipping*, a technique that limits the magnitude of gradients during training and improves optimization stability. Despite such improvements, preserving information over long sequences remained a fundamental challenge, which motivated the development of more advanced recurrent architectures such as Long Short-Term Memory (LSTM) [15] networks and Gated Recurrent Units (GRUs) [7, 8].

2.2 Long-Short Term Memory

The **Long Short-Term Memory (LSTM)** architecture, introduced by Hochreiter and Schmidhuber [15] in the 1990s, was proposed to address the limitations of standard RNNs when learning long-range dependencies. The key innovation of the architecture is the introduction of a dedicated memory state that can preserve information across many time steps. Instead of relying solely on the hidden state, the LSTM uses a set of gating mechanisms to control how information is written to, stored in, and retrieved from memory.

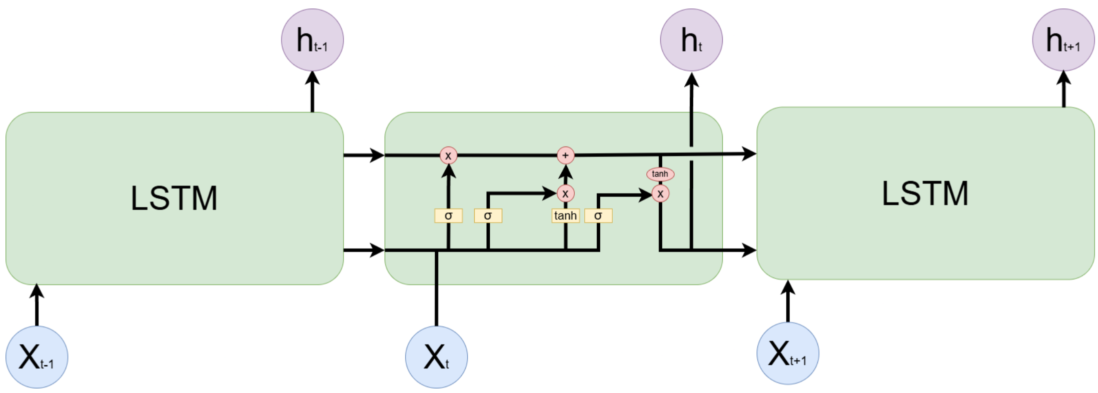


Figure 2: The LSTM cell, figure inspired by [20]

The standard LSTM cell maintains an internal cell state, c_t(Eq: 6), which acts as a long-term memory buffer, and a hidden state, h_t (Eq: 8), which serves as the short-term working memory routed to subsequent layers. The flow of information inside the cell is controlled by three gates with different purposes. The forget gate(Eq: 3) manages what proportion of the cell state to discard by analyzing the current input and the previous hidden state. The input gate(Eq: 4) controls how much new information, computed from the current input and previous hidden state, is incorporated into the cell state. Finally, the output gate(Eq: 7) determines which information stored in the updated cell state is propagated as the next hidden state.

$$f_t = \sigma(W_f x_t + R_f h_{t-1} + b_f) \tag{3}$$

$$i_t = \sigma(W_i x_t + R_i h_{t-1} + b_i) \tag{4}$$

$$\tilde{c}_t = \tanh(W_c x_t + R_c ht - 1 + b_c) \tag{5}$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \tag{6}$$

$$o_t = \sigma(W_o x_t + R_o h_{t-1} + b_o) \tag{7}$$

$$h_t = o_t \odot \tanh(c_t) \tag{8}$$

Overall, the introduction of a dedicated memory cell and gating mechanisms allows LSTMs to address the limitations of standard RNNs in capturing long-range dependencies. However, the sequential processing over the time dimension still limits parallelization and makes them computationally expensive for long sequences. Several architectural extensions were later proposed to improve their effectiveness in different settings. Bidirectional recurrent networks [29] improved contextual understanding by processing sequences in both forward and backward directions, while encoder-decoder architectures [32] enabled sequence-to-sequence learning by separating the encoding and decoding processes. In addition, stacked recurrent architectures [12] increased model capacity by composing multiple recurrent layers, allowing for more expressive hierarchical representations.

Despite these improvements, these variants did not remove the fundamental sequential bottleneck of recurrent computation, which limited scalability on long sequences. These limitations motivated the transition toward architectures that avoid recurrence entirely, most notably Transformer[33] models based on self-attention. In this broader context, recent work has revisited recurrent memory mechanisms at scale, leading to architectures such as xLSTM, which aim to combine the efficiency of structured recurrence with improved scalability and expressive memory modeling.

2.3 Extended Long-Short Term Memory

The extended LSTM (xLSTM) architecture was introduced by Beck et al. [4] to overcome these bottlenecks by leveraging modern techniques while fundamentally modifying the LSTM memory structure.

2.3.1 Addressing Classic LSTM Limitations

To understand the architectural shifts in the xLSTM, it is necessary to outline the three main bottlenecks of the classic LSTM:

1. **Inability to revise storage decisions:** Traditional LSTMs struggle to update or revise a stored value when new, more relevant information is found.

2. **Limited storage capacities:** Information in a classic LSTM is compressed into a scalar cell state, leading to poor performance when predicting rare tokens or retaining extensive context.

3. **Lack of parallelizability:** The recurrent memory update relies on hidden-to-hidden connections between consecutive time steps, requiring strictly sequential

processing and preventing the highly parallel computation that characterizes Transformer architectures.

To address these issues, xLSTM introduces several changes to the classical LSTM architecture. First, it replaces the traditional logistic gating mechanism with *exponential gating*, which provides a less restricted range for controlling memory updates. To ensure numerical stability during training and inference, xLSTM uses normalization and stabilization techniques. In addition, the architecture looks to improve the design of the memory state, resulting in two distinct recurrent building blocks: the *scalar LSTM (sLSTM)* and the *matrix LSTM (mLSTM)*. While sLSTM extends the traditional LSTM memory formulation with improved gating and memory management, mLSTM introduces a matrix-based memory structure capable of storing richer associations between tokens.

2.3.2 sLSTM

The sLSTM (scalar LSTM) remains structurally closer to the classic architecture but introduces exponential activation functions for the input and forget gates. This modification allows the model to control its storage more effectively:

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \mathbf{z}_t \qquad \text{cell state} \qquad (9)$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tilde{\mathbf{h}}_t, \qquad \tilde{\mathbf{h}}_t = \psi(\mathbf{c}_t) \qquad \text{hidden state} \qquad (10)$$

$$\mathbf{z}_t = \varphi(\tilde{\mathbf{z}}_t), \qquad \tilde{\mathbf{z}}_t = W_z x_t + R_z h_{t-1} + b_z \qquad \text{cell input} \qquad (11)$$

$$\mathbf{i}_t = \sigma(\tilde{\mathbf{i}}_t), \qquad \tilde{\mathbf{i}}_t = W_{\mathbf{i}} x_t + R_{\mathbf{i}} h_{t-1} + b_{\mathbf{i}} \qquad \text{input gate} \qquad (12)$$

$$\mathbf{f}_t = \sigma(\tilde{\mathbf{f}}_t), \qquad \tilde{\mathbf{f}}_t = W_{\mathbf{f}} x_t + R_{\mathbf{f}} h_{t-1} + b_{\mathbf{f}} \qquad \text{forget gate} \qquad (13)$$

$$\mathbf{o}_t = \sigma(\tilde{\mathbf{o}}_t), \qquad \tilde{\mathbf{o}}_t = W_{\mathbf{o}} x_t + R_{\mathbf{o}} h_{t-1} + b_{\mathbf{o}} \qquad \text{output gate} \qquad (14)$$

To prevent the numerical overflows specific to the exponential function, a normalizer state

(n_t) and stabilization states are utilised:

$$m_t = \max\left(\log(f_t) + m_{t-1}, \log(i_t)\right) \qquad \text{stabilizer state} \qquad (15)$$

$$i'_t = \exp\left(\log(i_t) - m_t\right) = \exp(\tilde{\imath}_t - m_t) \qquad \text{stabil. input gate} \qquad (16)$$

$$f'_t = \exp\left(\log(f_t) + m_{t-1} - m_t\right) \qquad \text{stabil. forget gate} \qquad (17)$$

Despite these changes, because it retains hidden-hidden recurrent connections, the sLSTM doesn't improve the sequential processing bottleneck of the classic LSTM. For the scope of this work, the architecture used in the experiments solely relies on mLSTM cells.

2.3.3 mLSTM

The mLSTM fundamentally shifts the LSTM paradigm to solve both the storage capacity and parallelization bottlenecks. To overcome the limited capacity of scalar states, the mLSTM expands the memory cell into a matrix $C \in \mathbb{R}^{d \times d}$ where d represents the embedding dimension of x_t. Crucially, to achieve this efficiency, the mLSTM entirely abandons memory mixing (the hidden-hidden recurrent connections) allowing parallelization.

$$C_t = \mathrm{f}_t C_{t-1} + \mathrm{i}_t v_t k_t^\top \qquad \text{cell state} \qquad (18)$$

$$n_t = \mathrm{f}_t n_{t-1} + \mathrm{i}_t k_t \qquad \text{norm. state} \qquad (19)$$

$$h_t = \mathbf{o}_t \odot \tilde{h}_t, \qquad \tilde{h}_t = C_t q_t / \max\left\{\left|n_t^\top q_t\right|, 1\right\} \qquad \text{hidden state} \qquad (20)$$

$$q_t = W_q x_t + b_q \qquad \text{query input} \qquad (21)$$

$$k_t = \frac{1}{\sqrt{d}} W_k x_t + b_k \qquad \text{key input} \qquad (22)$$

$$v_t = W_v x_t + b_v \qquad \text{value input} \qquad (23)$$

$$\mathrm{i'}_t = \exp(\tilde{i}_t - m_t), \qquad \tilde{i}_t = w_\mathrm{i}^\top x_t + b_\mathrm{i} \qquad \text{input gate} \qquad (24)$$

$$\mathrm{f'}_t = \exp\left(\log(\sigma(\tilde{f}_t)) + m_{t-1} - m_t\right), \qquad \tilde{f}_t = w_\mathrm{f}^\top x_t + b_\mathrm{f} \qquad \text{forget gate} \qquad (25)$$

$$\mathbf{o}_t = \sigma(\tilde{\mathbf{o}}_t), \qquad \tilde{\mathbf{o}}_t = W_\mathbf{o} x_t + b_\mathbf{o} \qquad \text{output gate} \qquad (26)$$

Relying on Bidirectional Associative Memory (BAM)[18] principles and Transformer terminology, the mLSTM stores information as key-value pairs (k_t, v_t) using a covariance (outer product) update rule, and retrieves it using a query vector (q_t). The mLSTM cell also uses the stabilizer state introduced in the sLSTM equations(Eq: 15)

3 Methodology

This section focuses on the technical details of the experiments that are presented in this work.

3.1 Research Objectives

1. **Baseline evaluation of the xLSTM-7B model**(Subsection 3.3): Reproduce the reported performance of xLSTM-7B and compare the obtained results with the reported performance of smaller xLSTM variants.

2. **Interpretability study**(Subsection 3.4): Dissect the xLSTM components and investigate the internal mechanisms responsible for memory formation, information retention, and representation development.

3. **Efficient Inference**(Subsection 3.5): evaluate whether the fixed-size memory state of xLSTM can be leveraged to reduce latency while maintaining task performance in a practical setup.

3.2 Models

3.2.1 xLSTM-7B

In this study, we use the pre-trained xLSTM-7B model available on Hugging Face. This implementation differs from the original xLSTM paper and uses a more transformer-like block structure. The main recurrent component is a multi-head mLSTM layer, shown in Figure ??. For each input representation x_t, the model applies Root Mean Square Normalization(RMSNorm)[37] and computes projections for the query, key, value, input gate, forget gate, and output gate. The query, key, value, input-gate, and forget-gate projections are reshaped into 8 heads, and each head is processed by an independent mLSTM cell with its own recurrent state. Each head produces an output of dimension 512 , giving a concatenated dimension of 4096 . The head outputs are normalized with MultiHead LayerNorm [3], concatenated, multiplied by the output gate, and linearly projected back to the model dimension before a residual addition at the xlstm block level.

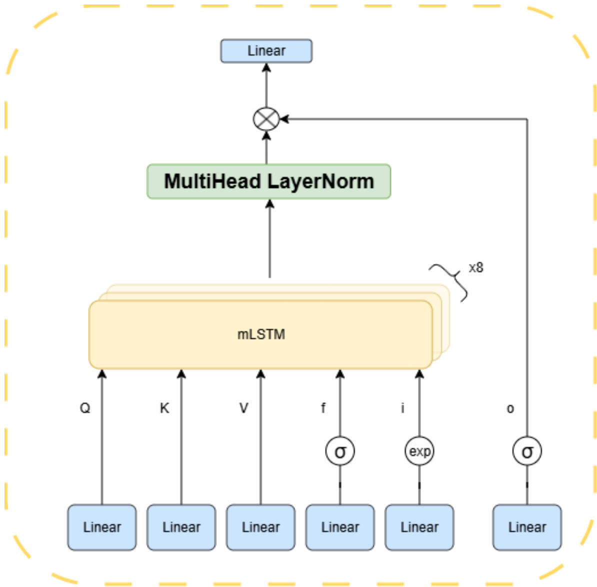


Figure 3: mLSTM layer - used in the xLSTM model

At the model level, these mLSTM blocks are arranged in a decoder-only language modeling architecture. Input tokens are first mapped to dense vector representations by an embedding layer. These representations are then processed by a stack of 32 xLSTM blocks, each containing the multi-head mLSTM sub-layer described above and a SwiGLU feed-forward sub-layer[30]. After the final block, the hidden representations are normalized with RMSNorm and projected into the vocabulary space using a linear output layer. Figure 4(Left) provides an overview of the complete model architecture.

The model has 6,865,424,896 parameters and uses an embedding dimension of 4096. Because the architecture is recurrent, it can in principle process sequences of arbitrary length. Instead of storing all previous token representations in a fixed attention window, the model carries information forward through the recurrent mLSTM states (C, n, m). In this study, the model is used in its pre-trained form, and no additional training or fine-tuning is performed.

3.2.2 Mistral-7B

For comparison purposes, we use the pretrained Mistral-7B-v0.3 model as a representative Transformer-based architecture. Like xLSTM, Mistral is a decoder-only language model and is used in its pretrained form without additional fine-tuning. However, the mechanism by which information is processed differs fundamentally between the two architectures.

Mistral relies on Grouped-Query Attention (GQA)[1] as its primary sequence modeling mechanism, utilizing 32 query heads and 8 key-value heads to optimize KV cache memory efficiency. The architecture uses Rotary Positional Embeddings (RoPE) [31] to sustain a 32,768-token context window. Within each Transformer decoder block, the embedded tokens are processed using RMS normalization before entering the GQA layer, where separate linear projections generate the query, key, and value vectors.

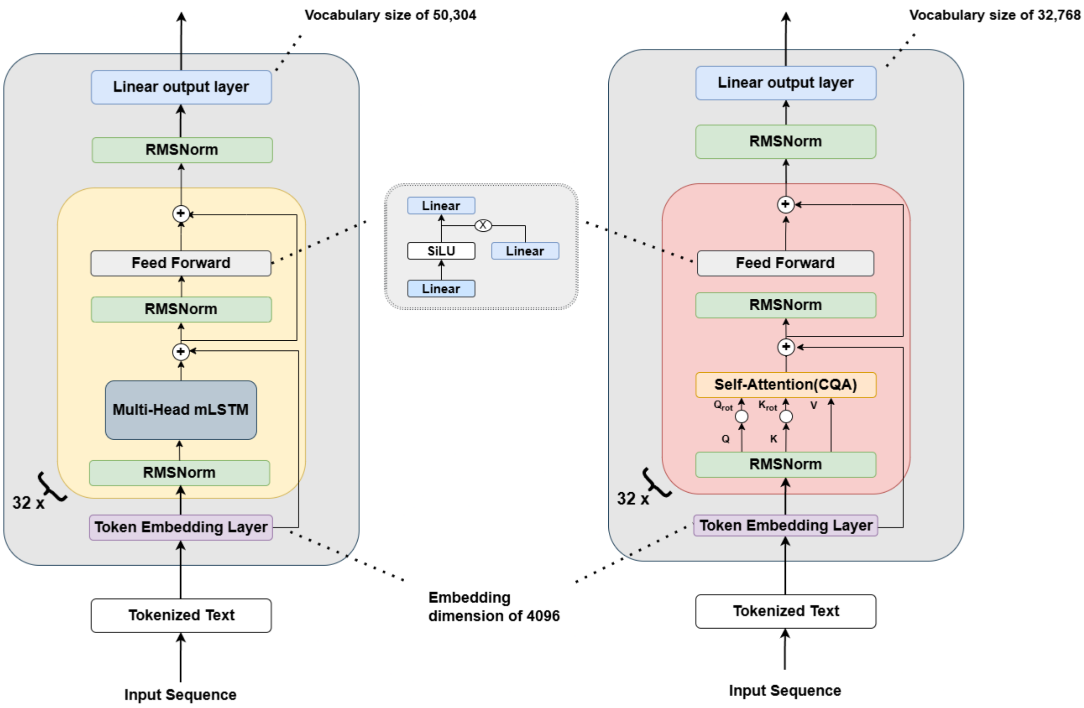


Figure 4: xLSTM and Mistral architectures. Left: xLSTM-7B. Right: Mistral-7B-v0.3. Adaptation after [26]

The attention outputs are combined, passed through a residual connection, and processed by a another RMS normalization layer. This is followed by a feed-forward network SwiGLU layer. A final residual connection completes the block.

The Mistral-7B architecture4(Right) consists of 32 Transformer decoder layers, each containing a multi-head self-attention module and a feed-forward network. The model uses a hidden dimension of 4096 and approximately 7 billion parameters, making it comparable in scale to the xLSTM-7B model. This architectural similarity in size allows differences observed in the experiments to be attributed primarily to the underlying sequence modeling mechanisms rather than to disparities in model capacity.

Throughout this study, Mistral serves as a reference model for both performance benchmarking and representation analysis, enabling a comparison between attention-based and recurrent approaches to language modeling.

3.3 Initial Benchmarking

To establish a baseline understanding of the model's capabilities, we evaluate xLSTM-7B on a collection of widely used language comprehension and reasoning benchmarks. These datasets assess different aspects of language modeling, ranging from commonsense reasoning and physical intuition to reading comprehension and scientific knowledge.

- **HellaSwag** [36]: A commonsense reasoning benchmark consisting of partially described situations followed by multiple candidate continuations. The task requires selecting the most plausible ending among several alternatives.
- **LAMBADA** [21]: A language modeling benchmark that evaluates the ability to predict the final word of a passage using the broader context of the document.
- **ARC-Challenge** [9]: The challenging subset of the AI2 Reasoning Challenge dataset. It consists of multiple-choice science questions that are difficult to answer using simple retrieval or pattern matching techniques. The benchmark evaluates

reasoning ability, scientific knowledge, and the capacity to combine information from multiple sources.

- **ARC-Easy** [9]: A simpler subset of the ARC benchmark containing elementary-level science questions.

- **PIQA** [6]: The Physical Interaction Question Answering benchmark evaluates physical commonsense reasoning. Questions describe everyday situations and require selecting the most plausible solution to a practical problem.

- **WinoGrande** [28]: A large-scale commonsense reasoning benchmark derived from the Winograd Schema Challenge. The task involves resolving ambiguous references in a sentence by selecting the correct antecedent.

3.4 Interpretability Study

The study of interpretable properties in deep neural networks has helped researchers better understand the internal mechanisms of blackbox models In computer vision, Zeiler et al. [35] improved the understanding of convolutional neural networks by using a deconvolutional framework to project intermediate feature activations back into the input space. This made it possible to observe how increasingly abstract visual representations emerge throughout the network layers.

In recurrent and Transformer-based architectures, interpretability research has focused on understanding what information is stored in hidden representations. Hewitt and Manning [14] showed that the hidden states of both RNNs and Transformers encode syntactic structure. Using a structural probing framework, they demonstrated that syntax trees can be recovered through a learned linear transformation of the representation space. Their results on models such as ELMo [24] and BERT [10] suggest that the model captures linguistic structure through its representations rather than storing it in an explicit form.

This interpretability study follows a layered approach that moves from high-level model outputs to internal mechanisms. We first analyze intermediate representations to understand what information is available at different depths of the network. We then compare these representations across architectures to study structural similarities and differences. Finally, we focus on the internal memory dynamics of xLSTM, examining how information is stored and controlled through its recurrent state and gating mechanisms.

3.4.1 The Logit Lens

The logit lens[19][2] is an experimental framework which studies the evolution of the latent representations in the network. To track how predictions form as text moves through the architecture, we extracted the hidden state vectors after each xLSTM block.

Under normal circumstances, word predictions are made at the very end of the network. In the logit lens setup, we take the extracted hidden states, normalize the values and use the language modeling head(learned linear layer) to project the output into the vocabulary space.

To evaluate how these internal guesses change and improve across depth, we analyzed the resulting word distributions in two ways:

- **Logits (Top-1 Predictions):** We evaluated the raw output scores across the entire vocabulary to identify which single token was the model's top choice at each individual block. This helps us visualize how the top choice token changes during the internal transformations within the network.

- **Target Ranks:** Even when an early layer does not select the final correct token as its top choice, that token may still be highly rated. We tracked the strict rank order of the ground truth prediction within each block's internal probability distribution. Monitoring how this rank climbs toward the first position shows how quickly the network narrows down its uncertainty.

3.4.2 Hidden State Similarity with CKA

Building on the logit lens analysis, which was originally tested on Transformer-based architectures, we next investigate how the representations learned by a recurrent architecture such as xLSTM compare to those of a Transformer model, specifically Mistral-7B. The reference model choice was made given that both models share the same embedding dimension and the same number of layers, which allows for a more aligned layer-wise analysis.

To evaluate differences in structural representations between the two architectures, we compare the hidden representations of each model using Centered Kernel Alignment (CKA) [17], a metric designed for measuring similarity between neural network representations. In this study, we use linear CKA computed over mean-centered hidden states across tokens, which computes the similarity using normalized Frobenius norms of cross-covariance matrices:

$$X_c = X - \frac{1}{B}\sum_{i=1}^{B} X_{i,\cdot} \tag{27}$$

$$Y_c = Y - \frac{1}{B}\sum_{i=1}^{B} Y_{i,\cdot} \tag{28}$$

where B represents the batch dimension

$$\text{Linear CKA}(X, Y) = \frac{\left\|Y_c^{\top} X_c\right\|_F^2}{\left\|X_c^{\top} X_c\right\|_F \left\|Y_c^{\top} Y_c\right\|_F} \tag{29}$$

For this purpose, we capture hidden state activations from each block of both models using the same input prompt, ensuring a consistent basis for comparison.

3.4.3 Gate Dynamics and Memory Formation

The update mechanism of the xLSTM memory state is controlled by the stabilized input and forget gates, $\tilde{i}l, t^{(h)}$ and $\tilde{f}l, t^{(h)}$, which determine how information is written to and retained in memory. To study these dynamics, we analyze the gate activations throughout the network. For each token position t, we compute the stabilized input and forget gate values across all heads and layers and average the results, yielding token-wise measures I_t and F_t. These quantities provide a global view of the rates of information writing and memory retention during sequence processing.

3.5 Context Memorization with Cell State Caching

The recurrent nature of xLSTM allows contextual information to be compressed into a fixed-size memory state. Unlike Transformer architectures, whose computational cost grows with the length of the input context, xLSTM maintains a constant-size cell state C_t of dimension $d \times d$, where d denotes the embedding dimension. This property motivates the exploration of memory-state caching, where the context is processed once and the resulting cell state is reused for subsequent queries.

The experiment requires a dataset containing contextual passages associated with multiple independent questions. To satisfy these requirements, we use the SQuAD dataset [25], which consists of Wikipedia passages grouped by title and context, each paired with several question-answer examples. On average the context consists of around 150-180 tokens per set of questions.

To ensure that the same context can be reused across multiple queries, we sample unique title-context pairs and retain a single context per title. Furthermore, only contexts associated with at least two questions are considered. The resulting evaluation set contains 102 question-answer pairs distributed across multiple shared contexts.

The benchmark is evaluated under two different configurations. In the first configuration, the model receives as input the concatenation of the context and the question for every query. Consequently, the entire context must be reprocessed each time a new question is asked.

In the second configuration, the context is processed only once. After the context has been consumed, the resulting cell states of the xLSTM are stored. Prior to answering each question associated with that context, the cached cell states are reloaded into the model, allowing inference to begin from the previously computed memory state rather than reprocessing the full context.

To provide a reference point, we also evaluate Mistral-7B on the same task. This comparison allows us to contrast the proposed cell-state caching mechanism of xLSTM with the key-value (KV) cache used by Transformer architectures. While both approaches aim to preserve previously processed information and avoid unnecessary computation, they use fundamentally different mechanisms. The objective of this experiment is therefore to evaluate the practical efficiency of these two approaches and determine how the recurrent memory of xLSTM compares to the KV-cache-based processing of a Transformer in terms of inference speed.

4 Implementation Details

The experiments in this study are implemented in Python using PyTorch and the Hugging Face Transformers library. The pretrained weights for both NX-AI/xLSTM-7B and mistralai/Mistral-7B-v0.3 are obtained directly from the Hugging Face model hub, ensuring a consistent and reproducible experimental setup.

For benchmark evaluation, we use the LM Evaluation Harness [11] framework as the primary tool for running standardized model assessments. Both this benchmark and the question-answering experiments require multiple inference operations, and for this reason we employ NVIDIA A100 GPUs with 40GB and 80GB of memory. The computation is executed in a cloud environment using the Modal infrastructure.

A significant part of the study relies on accessing internal model representations. For all experiments involving hidden states, we employ PyTorch forward hooks to extract intermediate activations during inference. This mechanism allows us to capture layer-wise representations without modifying the original model architecture.

When analyzing the cell states and input/forget gates, these values were not directly exposed by the model API. To address this limitation, we reconstruct them by intercepting the hidden states before and after each xLSTM block using hooks, and computing the corresponding values using the mathematical formulations provided in the xLSTM paper, as well as the reference implementation in the Hugging Face Transformers codebase.

In the experimental setup involving memory caching, we build upon an existing xLSTM caching implementation provided in the model repository, adapting it to support our usecase. This allows us to reuse previously computed memory states during question answering, reducing redundant computation during inference. We restrict decoding to a maximum of 25 new tokens and use a temperature of 1.0 to ensure consistent sampling behavior across runs. The limit of 25 generated tokens is chosen because the SQuAD dataset primarily requires short, span-like answers, making longer generations unnecessary for the task. Inference time is measured by aggregating per-step execution intervals from the start to the end of generation.

Finally, it is important to note that the xLSTM implementation benefits from optimized custom Triton kernels, which provide a significant speedup compared to standard PyTorch kernels. This optimization has a direct impact on overall runtime performance across all experiments.

5 Results and Observations

In this section we present and discuss the results for the experiments described in chapter 3.

5.1 Initial Benchmarking

In the original xLSTM paper, the authors evaluate smaller variants of the architecture namely 125M, 350M, 760M, and 1.3B parameters respectively, trained on SlimPajama (300B tokens). However, the prompting setup (e.g., zero-shot or few-shot) is not specified for these models. Table 1 reports their performance and compares it to the 7B model, whose results are reported in a zero-shot setting across all datasets.

Model	**LAMBADA ppl** $\downarrow$	**LAMBADA acc** $\uparrow$	**HellaSwag acc** $\uparrow$	**PIQA acc** $\uparrow$	**ARC-E acc** $\uparrow$	**ARC-C acc** $\uparrow$	**WinoGrande acc** $\uparrow$
xLSTM-125M	25.98	36.52	36.74	65.61	47.81	24.83	51.85
xLSTM-350M	11.49	49.33	48.06	69.59	55.72	26.62	54.38
xLSTM-760M	8.09	54.78	55.72	72.69	62.75	**32.59**	58.17
xLSTM-1.3B	**6.86**	**57.83**	**60.91**	**74.59**	**64.31**	**32.59**	**60.62**
xLSTM-7B	190.18	23.02	41.28	68.23	47.43	25.26	52.64

Table 1: Task-Specific Performance Metrics for xLSTM at different sizes

The xLSTM-7B results stand out as inconsistent with the trend observed in the smaller models. Across all reported benchmarks, it underperforms the 1.3B variant, despite having substantially more parameters.

The Hugging Face model card reports that the xLSTM-7B checkpoint was trained on approximately 2.3 trillion tokens of high-quality data derived from DCLM. The authors evaluate the model on a range of benchmark datasets using the LightEval framework. To verify these results, we reproduce a subset of the reported benchmarks using the LM Evaluation Harness. The comparison is presented in Table 2.

Model	Arc-Challenge (25-shot)	Hellaswag (10-shot)	Winogrande (5-shot)	PiQA (5-shot)
xLSTM-7B (HF)	**0.584**	**0.710**	**0.742**	**0.817**
xLSTM-7B (Reproduced)	0.418	0.625	0.565	0.736

Table 2: Benchmark results: reported vs reproduced

The scores obtained in this evaluation are consistently lower than those reported for the original checkpoint. Several factors may contribute to this discrepancy. Firstly, the evaluations were performed using different benchmarking frameworks, namely LightEval and LM Evaluation Harness, which may implement task formatting and evaluation procedures differently. Second, differences in prompting strategies, generation settings, or preprocessing steps can influence the final scores.

Despite these discrepancies, the reproduced results provide a perspective of this xLSTM-7B version. As the following experiments are conducted on this exact model, these results help contextualize the behavior observed in the following observations.

5.2 Interpretability Study

5.2.1 The logit lens

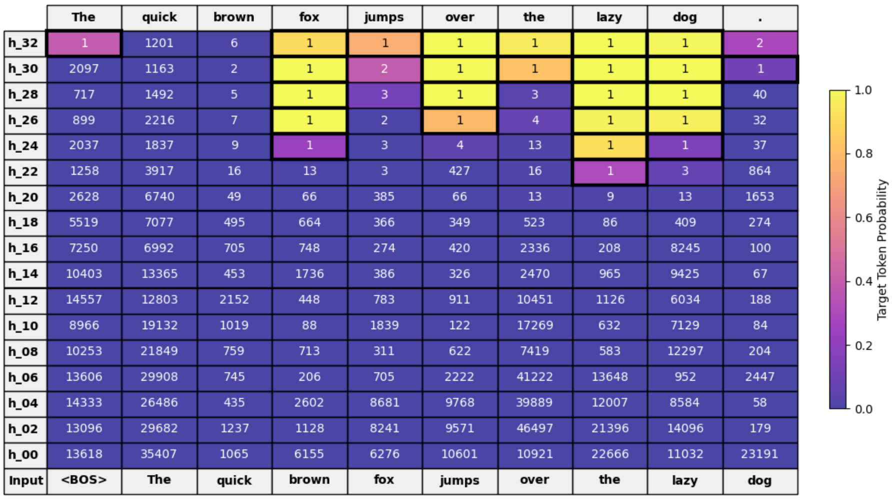
The figures related to logit lens follow a common structure: the columns correspond to hidden state indices, starting from the initial embedding layer and ending with the final representation produced by the last xLSTM block. For each column, the token shown in the bottom row represents the input token at time step t, while the token displayed in the top row corresponds to the ground-truth token at time step $t+1$.

In Figure 5, each cell contains the top prediction obtained by projecting the corresponding hidden state into the vocabulary space. The heatmap indicates the confidence assigned to the predicted token, while cells highlighted with a bold border denote predictions that match the ground-truth token.

Figure 5: Top token prediction per hidden state

From this visualization, we observe the expected behavior that confidence in the final prediction generally increases with network depth. In several cases, the correct token is already identified before the final layers, suggesting that later blocks primarily refine and strengthen representations that have already emerged in earlier stages of processing. A notable observation is the importance of the final layers in the prediction process. For many intermediate hidden states, the model assigns relatively high confidence (approximately 50–60%) to incorrect predictions. However, within the last four to six blocks, the predictions often shift toward either the correct token or an alternative prediction with very high confidence, frequently exceeding 90%. This behavior indicates that the final layers play a critical role in transforming intermediate representations into the model's final output distribution.

While Figure 5 allows us to observe the evolution of hidden representation quality, it's only limited to the top prediction. A question to answer is where the ground truth token ranks among the predictions for each layer. Figure 6 shows exactly that. We can see that after each layer the sentence representation improves the understanding of our model. Progressively, the ground-truth prediction climbs the ranks and at the final layer, oftenly appears to be among the top candidates.



	The	quick	brown	fox	jumps	over	the	lazy	dog	.
h_32	1	1201	6	1	1	1	1	1	1	2
h_30	2097	1163	2	1	2	1	1	1	1	1
h_28	717	1492	5	1	3	1	3	1	1	40
h_26	899	2216	7	1	2	1	4	1	1	32
h_24	2037	1837	9	1	3	4	13	1	1	37
h_22	1258	3917	16	13	3	427	16	1	3	864
h_20	2628	6740	49	66	385	66	13	9	13	1653
h_18	5519	7077	495	664	366	349	523	86	409	274
h_16	7250	6992	705	748	274	420	2336	208	8245	100
h_14	10403	13365	453	1736	386	326	2470	965	9425	67
h_12	14557	12803	2152	448	783	911	10451	1126	6034	188
h_10	8966	19132	1019	88	1839	122	17269	632	7129	84
h_08	10253	21849	759	713	311	622	7419	583	12297	204
h_06	13606	29908	745	206	705	2222	41222	13648	952	2447
h_04	14333	26486	435	2602	8681	9768	39889	12007	8584	58
h_02	13096	29682	1237	1128	8241	9571	46497	21396	14096	179
h_00	13618	35407	1065	6155	6276	10601	10921	22666	11032	23191
Input	<BOS>	The	quick	brown	fox	jumps	over	the	lazy	dog

Figure 6: The rank of the ground truth token per hidden state

5.2.2 Hidden State Comparison with Mistral-7B

After concluding the logit lens experiment, we observe similar behaviour between the xLSTM and what a Transformer would exhibit. Figure 7 displays a similarity map between the hidden representations of xLSTM-7B and Mistral-7B.

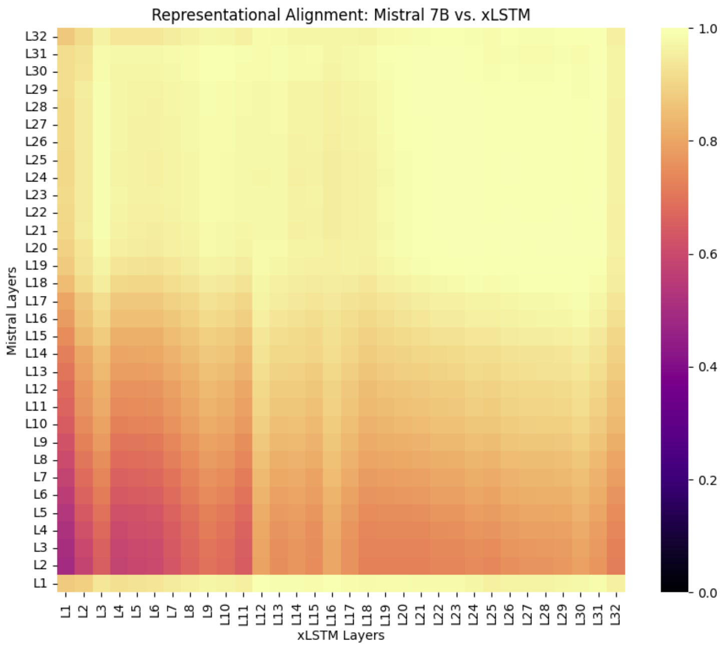


Figure 7: Representational alignment between the hidden states of xLSTM-7B and Mistral-7B

The CKA heatmap shows that the relationship between xLSTM and Mistral-7B representations is not aligned in a simple layer-to-layer way. Instead of a clean diagonal mapping, similarity is spread across multiple layers, meaning depth in one model does not correspond directly to depth in the other. Early layers in both models are relatively close to the input representation, but the overall structure diverges because Mistral processes tokens in parallel via attention, while xLSTM builds representations sequentially, leading to different rates of abstraction and organization.

Despite this mismatch in processing mechanisms, there is a strong convergence in representation at deeper stages. Early-to-middle layers show reduced and irregular similarity, reflecting the temporary divergence in how information representation evolves. However, from mid-to-late layers onward, similarity increases substantially, suggesting that both models gradually settle into a shared high-level representation space. This indicates that although their internal computation differs, both architectures ultimately encode similar semantic information at depth, even if that convergence is not aligned layer-by-layer.

5.2.3 Gate Dynamics and Memory Formation

In order to get a better understanding of the memory structure, namely the cell state C_t inside the mLSTM cell, in this experiment we test the sensibility of the input and forget gate values when processing a token at time t. The model was fed a passage from War and Peace by Tolostoy, which consists of 173 tokens.

Figure 8 highlights how much each token contributes to the model's memory. We can observe a progressive accumulation of context, with inital tokens triggering very low write activations, which steadilty ramp up as the details accumulate. Another observation is that identifiers such as proper nouns, or dates(e.g ”Vasili”, ”1805”) trigger the highest write values, showing that the model explicitly prioritizes embedding factual entities into its long-term memory state.

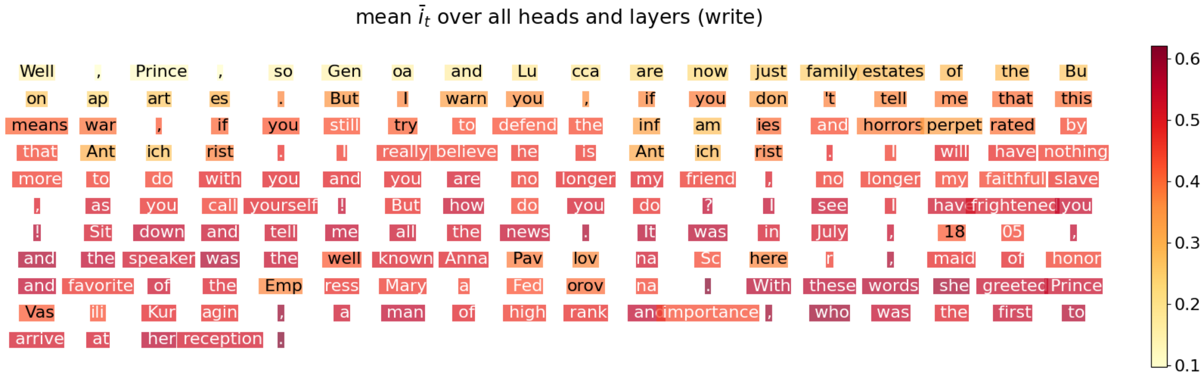


Figure 8: Input gate values per token

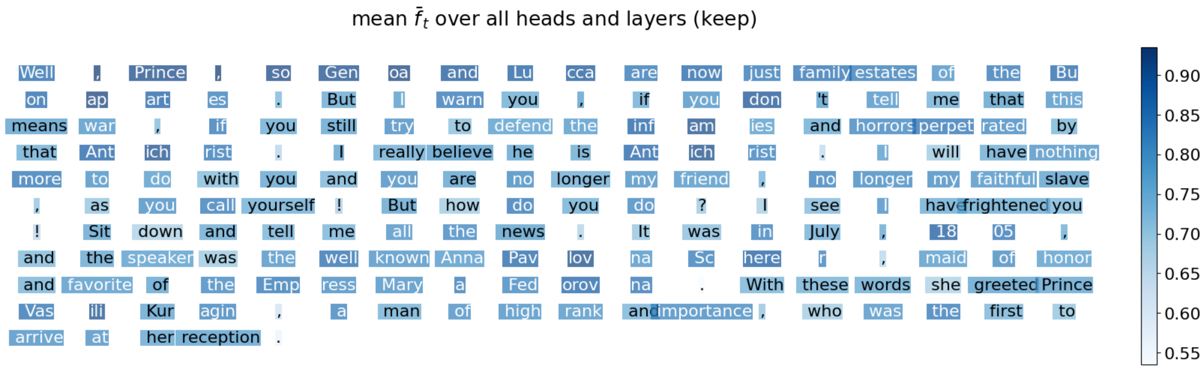


Figure 9: Forget gate values per token

Figure 8 we observe how the forget gate is strognly biased toward long-term memory retention. The gate values remain consistently high (mostly 0.7-0.9+), showing minimal forgetting compared to classical LSTMs, where memory is frequently reset. This suggests the model is optimized to preserve historical context across long token sequences, enabling stable retention of early information over extended spans.

A common observation for both figures is that gating activations remain consistent accross the tokens of individual token(e.g "Bu", "on", "ap", "art", "es"), indicating the architecture succsesfully treats these sub-word chunks as unified semantic blocks. Local variations appear at syntactic boundaries: punctuation tokens (commas, periods, etc.) indicating soft segmentation points where the model partially relaxes its memory state to transition between clauses or sentences.

As a complementary analysis, we also monitor the magnitude of the matrix-valued cell state $C_{l,t}^{(h)}$ by computing its Frobenius norm. The resulting plots are included in Appendix **??** and are used to relate gate activations to the overall evolution of the memory state.

5.3 Context Memorization with Cell State Caching

For this experiment we evalate the peformance of the xLSTM on a question answering task, exploring the possibility of caching the memory cells in order to boost the generation speed, while also keeping up the performance. To provide an additional reference point, we also evaluate Mistral-7B on the same dataset. Table 3 reports the corresponding results.

Model	Exact Match (%)	F1	Total Time (s)	Avg Time per Sample (s)
Mistral-7B	**2.94**	**30.50**	98.68	0.97
xLSTM-7B (no cache)	0.00	10.94	237.13	2.32
xLSTM-7B (with cache)	0.00	10.94	**86.93**	**0.85**

Table 3: Evaluation results comparing Mistral-7B and xLSTM-7B variants.

The results show a clear trade-off between accuracy and efficiency, with one model clearly balancing the best both of them. Mistral-7B is the strongest in terms of quality. It achieves the highest exact match (2.94%) and a substantially higher F1 score (30.50) compared to both xLSTM variants. Even though the exact match is still low in absolute terms, the gap in F1 indicates that Mistral is producing more partially correct or structurally similar outputs, not just exact answers.

Both xLSTM variants collapse to the same quality level: 0% exact match and an identical F1 of 10.94. This suggests that caching does not affect output correctness, which is expected, but more importantly that the underlying model behavior is unchanged across the two runs. The fact that both are significantly below Mistral indicates a meaningful gap in modeling capability or adaptation to the task distribution.

Where xLSTM becomes interesting is efficiency. Without cache, it is the slowest by a large margin (2.32s vs 0.97s for Mistral), making it clearly less efficient in a naive inference setup. However, with cache, generation time drops to 0.85s per sample, making it faster than Mistral and representing roughly a $2.7\times$ speedup compared to the no-cache version. This confirms that the caching mechanism is effective in reducing computation cost without trading off performance.

An additional observation is that the benefit of caching is expected to scale with context length. As the sequence grows, models without caching have to process increasingly expensive recomputation of the whole context. This is especially useful in practical applications involving long prompts, multi-turn interaction histories, or retrieval-augmented contexts.

6 Conclusion and Future Work

6.1 General Observations

Overall, the model demonstrates promising learning behavior and efficient long-context processing. While it does not yet match transformer performance in accuracy, it exhibits clear advantages in latency and scalability for long-context inference scenarios.

The reproduced benchmark results did not fully match the reported performance, likely due to missing implementation details or differences in evaluation frameworks. This highlights the sensitivity of reported results to evaluation setup and preprocessing choices.

Logit lens analysis across stacked recurrent layers reveals behavior similar to transformer architectures, where intermediate representations progressively align with final predictions. Hidden state comparisons with Mistral-7B show strong convergence in learned representations, suggesting that xLSTM develops internal structures closely aligned with transformer-style language representations despite architectural differences. Additionally, The model effectively identifies and prioritizes important tokens within the sequence, attributing stronger updates to meaningful elements such as entities and key contextual markers. The input and forget gates work in a coordinated way to preserve relevant information while selectively integrating new content into the memory state.

Memory caching in a question-answering setup yields up to a 2.7× speedup in generation time by reusing precomputed memory states and avoiding redundant computation over long contexts. The speedup increases with context length, making the approach particularly effective for long-sequence inference. However, performance still remains below a comparable transformer baseline (Mistral-7B) in terms of quality metrics.

6.2 Limitations

Despite the promising results, the model underperforms compared to strong transformer baselines across several benchmarks. In addition, experimental work was constrained by engineering limitations, including difficulties in deploying Triton kernels across different operating systems.

6.3 Future Work

This study represents an initial exploration of the model's capabilities. Future work should extend evaluation to longer sequences and include benchmarks that explicitly test long-range memory and contextual retention. This would provide a more comprehensive assessment of the model's strengths and limitations in long-context reasoning settings.

References

[1] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. *GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints*. 2023. arXiv: 2305.13245 [cs.CL]. URL: https://arxiv.org/abs/2305.13245.

[2] Belrose et al. "Eliciting Latent Predictions from Transformers with the Tuned Lens". In: *arXiv preprint arXiv:2303.08112* (2023).

[3] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. *Layer Normalization*. 2016. arXiv: 1607.06450 [stat.ML]. URL: https://arxiv.org/abs/1607.06450.

[4] Maximilian Beck, Korbinian Pöppel, Markus Spanring, Andreas Auer, Oleksandra Prudnikova, Michael Kopp, Günter Klambauer, Johannes Brandstetter, and Sepp Hochreiter. *xLSTM: Extended Long Short-Term Memory*. 2024. arXiv: 2405.04517 [cs.LG]. URL: https://arxiv.org/abs/2405.04517.

[5] Yoshua Bengio, Patrice Simard, and Paolo Frasconi. "Learning Long-Term Dependencies with Gradient Descent is Difficult". In: *IEEE Transactions on Neural Networks* (1994).

[6] Yonatan Bisk, Rowan Zellers, Ronan Le Bras, Jianfeng Gao, and Yejin Choi. "Piqa: Reasoning about physical commonsense in natural language". In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 34. 05. 2020, pp. 7432–7439.

[7] Kyunghyun Cho, Bart van Merrienboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. *Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation*. 2014. arXiv: 1406.1078 [cs.CL]. URL: https://arxiv.org/abs/1406.1078.

[8] Junyoung Chung, Caglar Gulcehre, KyungHyun Cho, and Yoshua Bengio. *Empirical Evaluation of Gated Recurrent Neural Networks on Sequence Modeling*. 2014. arXiv: 1412.3555 [cs.NE]. URL: https://arxiv.org/abs/1412.3555.

[9] Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. "Think you have solved question answering? try arc, the ai2 reasoning challenge". In: *arXiv preprint arXiv:1803.05457* (2018).

[10] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding". In: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*. Ed. by Jill Burstein, Christy Doran, and Thamar Solorio. Minneapolis, Minnesota: Association for Computational Linguistics, June 2019, pp. 4171–4186. DOI: `10.18653/v1/N19-1423`. URL: `https://aclanthology.org/N19-1423/`.

[11] Leo Gao, Jonathan Tow, Baber Abbasi, Stella Biderman, Sid Black, Anthony DiPofi, Charles Foster, Laurence Golding, Jeffrey Hsu, Alain Le Noac'h, Haonan Li, Kyle McDonell, Niklas Muennighoff, Chris Ociepa, Jason Phang, Laria Reynolds, Hailey Schoelkopf, Aviya Skowron, Lintang Sutawika, Eric Tang, Anish Thite, Ben Wang, Kevin Wang, and Andy Zou. *The Language Model Evaluation Harness*. Version v0.4.3. July 2024. DOI: `10.5281/zenodo.12608602`. URL: `https://zenodo.org/records/12608602`.

[12] Alex Graves. "Generating Sequences With Recurrent Neural Networks". In: *arXiv preprint arXiv:1308.0850* (2013).

[13] Albert Gu and Tri Dao. *Mamba: Linear-Time Sequence Modeling with Selective State Spaces*. 2024. arXiv: `2312.00752 [cs.LG]`. URL: `https://arxiv.org/abs/2312.00752`.

[14] John Hewitt and Christopher D. Manning. "A Structural Probe for Finding Syntax in Word Representations". In: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*. Ed. by Jill Burstein, Christy Doran, and Thamar Solorio. Minneapolis, Minnesota: Association for Computa-

tional Linguistics, June 2019, pp. 4129–4138. DOI: 10.18653/v1/N19-1419. URL: https://aclanthology.org/N19-1419/.

[15] Sepp Hochreiter and Jürgen Schmidhuber. "LSTM can solve hard long time lag problems". In: *Proceedings of the 10th International Conference on Neural Information Processing Systems*. NIPS'96. Denver, Colorado: MIT Press, 1996, pp. 473–479.

[16] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, Lélio Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. *Mistral 7B*. 2023. arXiv: 2310.06825 [cs.CL]. URL: https://arxiv.org/abs/2310.06825.

[17] Simon Kornblith, Mohammad Norouzi, Honglak Lee, and Geoffrey Hinton. *Similarity of Neural Network Representations Revisited*. 2019. arXiv: 1905.00414 [cs.LG]. URL: https://arxiv.org/abs/1905.00414.

[18] Bart Kosko. "Adaptive bidirectional associative memories". In: *Applied optics* 26.23 (1987), pp. 4947–4960.

[19] nostalgebraist. *Interpreting GPT: The Logit Lens*. 2020. URL: https://www.lesswrong.com/posts/AcKRB8wDpdaN6v6ru/interpreting-gpt-the-logit-lens.

[20] Christopher Olah. *Understanding LSTM Networks*. 2015. URL: https://colah.github.io/posts/2015-08-Understanding-LSTMs/ (visited on 06/01/2026).

[21] Denis Paperno, Germán Kruszewski, Angeliki Lazaridou, Ngoc Quan Pham, Raffaella Bernardi, Sandro Pezzelle, Marco Baroni, Gemma Boleda, and Raquel Fernández. "The LAMBADA dataset: Word prediction requiring a broad discourse context". In: *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 2016, pp. 1525–1534.

[22] Razvan Pascanu, Tomas Mikolov, and Yoshua Bengio. "On the Difficulty of Training Recurrent Neural Networks". In: *ICML*. 2013.

[23] Bo Peng, Eric Alcaide, Quentin Anthony, Alon Albalak, Samuel Arcadinho, Stella Biderman, Huanqi Cao, Xin Cheng, Michael Chung, Matteo Grella, Kranthi Kiran GV, Xuzheng He, Haowen Hou, Jiaju Lin, Przemyslaw Kazienko, Jan Kocon, Jiaming Kong, Bartlomiej Koptyra, Hayden Lau, Krishna Sri Ipsit Mantri, Ferdinand Mom, Atsushi Saito, Guangyu Song, Xiangru Tang, Bolun Wang, Johan S. Wind, Stanislaw Wozniak, Ruichong Zhang, Zhenyuan Zhang, Qihang Zhao, Peng Zhou, Qinghua Zhou, Jian Zhu, and Rui-Jie Zhu. *RWKV: Reinventing RNNs for the Transformer Era*. 2023. arXiv: 2305.13048 [cs.CL]. URL: https://arxiv.org/abs/2305.13048.

[24] Matthew E. Peters, Mark Neumann, Mohit Iyyer, Matt Gardner, Christopher Clark, Kenton Lee, and Luke Zettlemoyer. *Deep contextualized word representations*. 2018. arXiv: 1802.05365 [cs.CL]. URL: https://arxiv.org/abs/1802.05365.

[25] Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. "SQuAD: 100,000+ Questions for Machine Comprehension of Text". In: *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*. Ed. by Jian Su, Kevin Duh, and Xavier Carreras. Austin, Texas: Association for Computational Linguistics, Nov. 2016, pp. 2383–2392. DOI: 10.18653/v1/D16-1264. arXiv: 1606.05250 [cs.CL]. URL: https://aclanthology.org/D16-1264.

[26] Sebastian Raschka. *LLM Architecture Gallery*. Accessed: 2026-06-10. Mar. 2026. URL: https://sebastianraschka.com/llm-architecture-gallery/.

[27] David E Rumelhart, Geoffrey E Hinton, and Ronald J Williams. "Learning representations by back-propagating errors". In: *nature* 323.6088 (1986), pp. 533–536.

[28] Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. "Winogrande: An adversarial winograd schema challenge at scale". In: *Communications of the ACM* 64.9 (2021), pp. 99–106.

[29] Mike Schuster and Kuldip K. Paliwal. "Bidirectional Recurrent Neural Networks". In: *IEEE Transactions on Signal Processing* (1997).

[30] Noam Shazeer. *GLU Variants Improve Transformer*. 2020. arXiv: 2002.05202 [cs.LG]. URL: https://arxiv.org/abs/2002.05202.

[31] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. *RoFormer: Enhanced Transformer with Rotary Position Embedding*. 2023. arXiv: 2104.09864 [cs.CL]. URL: https://arxiv.org/abs/2104.09864.

[32] Ilya Sutskever, Oriol Vinyals, and Quoc V. Le. "Sequence to Sequence Learning with Neural Networks". In: *NeurIPS* (2014).

[33] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. "Attention is all you need". In: *Advances in neural information processing systems* 30 (2017).

[34] Paul J. Werbos. "Backpropagation Through Time: What It Does and How to Do It". In: *Proceedings of the IEEE* (1990).

[35] Matthew D Zeiler and Rob Fergus. *Visualizing and Understanding Convolutional Networks*. 2013. arXiv: 1311.2901 [cs.CV]. URL: https://arxiv.org/abs/1311.2901.

[36] Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. "HellaSwag: Can a Machine Really Finish Your Sentence?" In: *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*. 2019, pp. 4791–4800.

[37] Biao Zhang and Rico Sennrich. *Root Mean Square Layer Normalization*. 2019. arXiv: 1910.07467 [cs.LG]. URL: https://arxiv.org/abs/1910.07467.